



Báo Cáo Bài Tập Lớn Môn Hệ Điều Hành

Nguyên lý hệ điều hành (Trường Đại học Mở Hà Nội)



Scan to open on Studeersnel

**BỘ GIÁO DỤC & ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ HÀ NỘI
KHOA ĐIỆN – ĐIỆN TỬ**

----- *** -----



**BÁO CÁO NHÓM 10
MÔN: HỆ ĐIỀU HÀNH**

Đề tài: Thiết kế và cài đặt một SHELL đơn giản (TinyShell)

Giảng viên hướng dẫn: Phạm Thị Thu Trang

Khóa: K26 – ĐT3

Sinh viên thực hiện:- Quách Đại Dũng

- Dương Ngô Đức Anh

- Nguyễn Nam Dương

LỜI CẢM ƠN

- Lời đầu tiên, em xin chân thành cảm ơn Trường Đại học Mở Hà Nội đã tạo điều kiện thuận lợi để em thực hiện tốt bài môn học Kỹ thuật mạch.
 - Đặc biệt, em xin chân thành cảm ơn ThS. Nguyễn Mạnh Hùng đã rất tận tình hướng dẫn, chỉ bảo em trong suốt thời gian thực hiện bài tập vừa qua.
 - Em cũng xin chân thành cảm ơn tất cả các thầy, cô giảng viên khoa Điện – Điện tử đã tận tình giảng dạy, trang bị cho em những kiến thức cần thiết, quý báu để giúp em thực hiện được bài tập được nêu trong môn học trên.
 - Em đã cố gắng hết sức, nhưng với trình độ còn hạn chế, trong quá trình thực hiện bài tập không tránh khỏi những thiếu sót. Em hi vọng sẽ nhận được những ý kiến nhận xét, góp ý của các thầy về những vấn đề triển khai trong báo cáo.
- Em xin trân trọng cảm ơn!

Hà Nội, ngày 23 tháng 09 năm 2025

Sinh viên thực hiện

Quách Đại Dũng

Dương Ngô Đức Anh

Nguyễn Nam Dương

Mục Lục:

1. Mở đầu

- a. Giới thiệu đề tài
- b. Mục tiêu
- c. Phạm vi nghiên cứu.....
- d. Cấu trúc báo cáo.....

2. Tổng quan về Shell và TinyShell

- a. Khái niệm Shell.....
- b. Các loại shell phổ biến.....
- c. TinyShell là gì ?
- d. Tại sao lại cần TinyShell ?

3. Thiết kế và phát triển TinyShell

- a. Kiến trúc tổng thể
- b. Cấu trúc dữ liệu
- c. Luồng xử lý chính
- d. Quản lý tiến trình
- e. Quản lý công việc nền
- f. Xử lý tín hiệu và sự kiện

4. Phân tích chi tiết Mã Nguồn

- a. Các thư viện và hằng số
- b. Các cấu trúc dữ liệu Job
- c. Các hàm tiện ích
- d. Xử lý lệnh tích hợp
- e. Quản lý tiến trình
- f. Khởi động và quản lý Jobs
- g. Hàm main

5. Các tính năng chính của TinyShell

- a. Thực thi lệnh cơ bản
- b. Quản lý Jobs
- c. Điều khiển tiến trình
- d. Các lệnh tích hợp
- e. Xử lý môi trường

6. Kết quả thực nghiệm và đánh giá

- a. Môi trường thử nghiệm
- b. Các tình huống thử nghiệm
- c. Kết quả đạt được
- d. Hạn chế và điểm cần cải thiện

7. Kết luận và hướng phát triển

- a. Tổng kết kết quả
- b. Bài học kinh nghiệm
- c. Hướng phát triển trong tương lai

8. Tài liệu tham khảo

- a. Sách và tài liệu
- b. Nguồn trực tuyến

1. MỞ ĐẦU

a. Giới thiệu đề tài

Hệ điều hành là một trong những thành phần cốt lõi của bất kỳ hệ thống máy tính nào, đóng vai trò là cầu nối giữa phần cứng và người dùng. Một trong những giao diện quan trọng nhất để tương tác với hệ điều hành là Shell – một chương trình thông dịch lệnh. Shell cho phép người dùng giao tiếp với nhân (kernel) của hệ điều hành thông qua các dòng lệnh, thực thi các chương trình và quản lý các tiến trình.

Trong bối cảnh đó, đề tài "Tiny Shell cho Hệ điều hành Windows" được thực hiện với mục đích xây dựng một phiên bản Shell tối giản, mô phỏng lại các chức năng cơ bản của các Shell phổ biến như Command Prompt (CMD) hay PowerShell trên nền tảng Windows. Việc tự tay triển khai một Shell không chỉ giúp củng cố kiến thức nền tảng về nguyên lý hoạt động của hệ điều hành mà còn mang lại cái nhìn sâu sắc về cách các tiến trình được tạo ra, quản lý và giao tiếp với nhau.

b. Mục tiêu

Đề tài tập trung vào việc hiện thực hóa các mục tiêu chính sau:

- Nghiên cứu và hiểu rõ cơ chế hoạt động của một trình thông dịch lệnh (Shell).
- Thiết kế và xây dựng một chương trình Tiny Shell có khả năng phân tích và thực thi các lệnh từ người dùng.
- Triển khai các tính năng cốt lõi, bao gồm: thực thi lệnh tiền cảnh (foreground), chạy các công việc nền (background), và quản lý các công việc này.
- Xử lý các lệnh tích hợp (built-in commands) cơ bản như `quit`, `jobs`, `fg`, `bg`.
- Áp dụng các lời gọi hệ thống (system calls) của Windows API để quản lý tiến trình và công việc.

c. Phạm vi nghiên cứu

Do giới hạn về thời gian và nguồn lực, dự án sẽ tập trung vào các phạm vi sau:

- Nền tảng: Hệ điều hành Windows.
- Ngôn ngữ lập trình: C/C++, sử dụng các thư viện chuẩn và Windows API.
- Chức năng:
 - Phân tích dòng lệnh đơn giản, không bao gồm các tính năng phức tạp như đường ống (pipe), chuyển hướng I/O (I/O redirection).
 - Thực thi các chương trình thực thi (file `.exe`).

- Quản lý các công việc (jobs) đang chạy ở chế độ nền.
- Hỗ trợ một số lệnh tích hợp cơ bản.

d. Cấu trúc báo cáo

Báo cáo được tổ chức thành các chương chính sau:

- Chương 1: Mở đầu: Giới thiệu tổng quan về đề tài, mục tiêu, phạm vi và cấu trúc báo cáo.
- Chương 2: Tổng quan về Shell và Tiny Shell: Trình bày các khái niệm cơ bản về Shell, các loại Shell phổ biến và định nghĩa về Tiny Shell.
- Chương 3: Thiết kế và Triển khai Tiny Shell: Mô tả kiến trúc tổng thể, luồng xử lý, và các cơ chế quản lý tiến trình, công việc.
- Chương 4: Phân tích chi tiết Mã nguồn: Đi sâu vào phân tích các thành phần chính trong mã nguồn của chương trình.
- Chương 5: Các tính năng chính của Tiny Shell: Giới thiệu và minh họa các chức năng đã được hiện thực.
- Chương 6: Kết quả thực nghiệm và Đánh giá: Trình bày môi trường, các kịch bản thử nghiệm và đánh giá kết quả.
- Chương 7: Kết luận và Hướng phát triển: Tổng kết lại các kết quả đạt được và đề xuất các hướng cải tiến trong tương lai.
- Tài liệu tham khảo và Phụ lục: Liệt kê các tài liệu đã tham khảo

2. TỔNG QUAN VỀ SHELL VÀ TINY SHELL

a. Khái niệm Shell

Shell là một chương trình máy tính đặc biệt cung cấp một giao diện cho người dùng để truy cập vào các dịch vụ của hệ điều hành. Nó hoạt động như một trình thông dịch dòng lệnh (Command-Line Interpreter - CLI), nhận lệnh từ người dùng, diễn giải chúng và yêu cầu hệ điều hành thực thi. Về bản chất, Shell là lớp vỏ (shell) bao bọc quanh nhân (kernel) của hệ điều hành, che đi sự phức tạp của việc tương tác trực tiếp với kernel.

Chức năng chính của Shell bao gồm:

- **Đọc và phân tích lệnh:** Nhận chuỗi văn bản từ người dùng và phân tách thành tên lệnh và các đối số.
- **Thực thi chương trình:** Tải và chạy các chương trình thực thi.
- **Quản lý môi trường:** Cung cấp và quản lý các biến môi trường cho các tiến trình.
- **Cung cấp tính năng lập trình kịch bản (scripting):** Cho phép người dùng viết các tệp kịch bản để tự động hóa các tác vụ.

b. Các loại Shell phổ biến

Có rất nhiều loại Shell khác nhau trên các hệ điều hành, mỗi loại có cú pháp và tính năng riêng.

- **Trên hệ điều hành Unix/Linux:**
 - **Bourne Shell (sh):** Một trong những Shell đầu tiên và có ảnh hưởng lớn.
 - **Bourne-Again Shell (bash):** Phiên bản cải tiến của **sh**, là Shell mặc định trên hầu hết các bản phân phối Linux và macOS.
 - **C Shell (csh):** Có cú pháp tương tự ngôn ngữ lập trình C.
 - **KornShell (ksh):** Kết hợp các tính năng tốt nhất của Bourne Shell và C Shell.
 - **Z Shell (zsh):** Một Shell hiện đại với nhiều tính năng nâng cao.
- **Trên hệ điều hành Windows:**
 - **Command Prompt (cmd.exe):** Trình thông dịch lệnh mặc định trên các phiên bản Windows cũ.
 - **PowerShell:** Một Shell mạnh mẽ hơn, được xây dựng trên nền tảng .NET Framework, cung cấp khả năng tự động hóa và quản lý hệ thống nâng cao.
 - **Windows Subsystem for Linux (WSL):** Cho phép chạy các môi trường Linux, bao gồm các Shell như **bash**, trực tiếp trên Windows.

c. Tiny Shell là gì?

Tiny Shell là một phiên bản rút gọn của một Shell truyền thống, được xây dựng cho mục đích học tập và nghiên cứu. Nó không nhằm mục đích thay thế các Shell đầy đủ tính năng mà tập trung vào việc hiện thực hóa các chức năng cốt lõi và cơ bản nhất. Thông qua việc xây dựng một Tiny Shell, người học có thể hiểu rõ hơn về:

- Quản lý tiến trình (tạo, kết thúc, tạm dừng).
- Xử lý tín hiệu (signal handling).
- Giao tiếp giữa các tiến trình.
- Cách một trình thông dịch lệnh hoạt động từ bên trong.

d. Tại sao cần Tiny Shell?

Việc phát triển một Tiny Shell mang lại nhiều lợi ích quan trọng trong quá trình học tập môn Nguyên lý Hệ điều hành:

- **Học đi đôi với hành:** Thay vì chỉ học lý thuyết suông, việc tự tay viết mã giúp sinh viên áp dụng kiến thức vào thực tế, hiểu sâu hơn về các khái niệm như [`fork\(\)`](#), [`exec\(\)`](#), [`waitpid\(\)`](#) (trên Unix) hay [`CreateProcess\(\)`](#), [`WaitForSingleObject\(\)`](#) (trên Windows).
- **Hiểu rõ kiến trúc hệ thống:** Cung cấp cái nhìn trực quan về mối quan hệ giữa ứng dụng người dùng, Shell và nhân hệ điều hành.
- **Phát triển kỹ năng lập trình hệ thống:** Rèn luyện kỹ năng làm việc với các API cấp thấp của hệ điều hành, xử lý lỗi và quản lý tài nguyên.
- **Nền tảng cho các dự án phức tạp hơn:** Kiến thức thu được từ việc xây dựng Tiny Shell là tiền đề vững chắc để phát triển các ứng dụng hệ thống phức tạp hơn sau này.

3. THIẾT KẾ VÀ TRIỂN KHAI TINY SHELL

a. Kiến trúc tổng thể

Kiến trúc của Tiny Shell được thiết kế theo một vòng lặp vô hạn, mô phỏng hoạt động của một Shell thực thụ. Sơ đồ kiến trúc có thể được mô tả như sau:

1. **Khởi tạo (Initialization):** Thiết lập môi trường, khởi tạo danh sách công việc (jobs list).
2. **Hiển thị dấu nhắc (Prompt):** Hiển thị một dấu nhắc lệnh (ví dụ: `tsh>`) để thông báo cho người dùng rằng Shell đã sẵn sàng nhận lệnh.
3. **Đọc lệnh (Read Command):** Đọc một dòng lệnh từ đầu vào chuẩn (bàn phím).
4. **Phân tích lệnh (Parse Command):** Phân tách dòng lệnh thành tên chương trình và các đối số. Xác định xem lệnh nên được chạy ở chế độ tiền cảnh hay nền.
5. **Thực thi lệnh (Execute Command):**
 - Nếu là lệnh tích hợp (built-in), thực thi trực tiếp trong tiến trình của Shell.
 - Nếu là chương trình bên ngoài, tạo một tiến trình con mới để thực thi chương trình đó.
6. **Quản lý tiến trình/công việc:**
 - Nếu là công việc tiền cảnh, Shell sẽ chờ cho đến khi nó hoàn thành.
 - Nếu là công việc nền, Shell sẽ thêm nó vào danh sách công việc và tiếp tục vòng lặp để nhận lệnh tiếp theo.
7. **Lặp lại:** Quay trở lại bước 2.

b. Cấu trúc dữ liệu

Để quản lý các công việc (jobs), một cấu trúc dữ liệu chính được sử dụng. Mỗi công việc tương ứng với một tiến trình (hoặc một nhóm tiến trình) được khởi chạy từ Shell.

```
struct job_t {  
    pid_t pid;           // Process ID (ID của tiến trình)  
    int jid;             // Job ID (ID của công việc, duy nhất trong Shell)  
    int state;           // Trạng thái: UNDEF, BG (nền), FG (tiền cảnh), ST (dừng)  
    char cmdline[MAXLINE]; // Dòng lệnh tương ứng  
};
```

Một mảng toàn cục của cấu trúc này (`struct job_t jobs[MAXJOBS];`) được sử dụng để lưu trữ tất cả các công việc đang hoạt động.

c. Luồng xử lý chính

Luồng xử lý chính của Tiny Shell nằm trong hàm main, tuân theo một vòng lặp while(1) đơn giản:

1. **Hiển thị dấu nhắc:** printf("tsh> ");
2. **Đọc dòng lệnh:** Sử dụng hàm fgets() để đọc toàn bộ dòng nhập liệu từ người dùng.
3. **Phân tích dòng lệnh:** Hàm parseline() sẽ phân tích chuỗi đầu vào, tách thành các token (tên lệnh và đối số) và xác định xem có phải là công việc nền hay không (dựa vào sự xuất hiện của ký tự &).
4. **Thực thi:**
 - Hàm builtin_cmd() được gọi để kiểm tra xem lệnh có phải là lệnh tích hợp (quit, jobs, bg, fg) không.
 - Nếu có, thực thi và tiếp tục vòng lặp.
 - Nếu không, gọi hàm do_bgfg() hoặc thực thi lệnh bằng cách tạo tiến trình mới.
5. **Tạo tiến trình mới (cho các lệnh không phải built-in):**
 - Sử dụng hàm CreateProcess() của Windows API để tạo một tiến trình con mới.
 - Tiến trình cha (Shell) sẽ dựa vào chế độ (nền/tiền cảnh) để quyết định hành động tiếp theo.

d. Quản lý tiến trình

Quản lý tiến trình là trái tim của Tiny Shell. Trên Windows, các hàm API sau được sử dụng:

- CreateProcess(): Tương đương với fork() và exec() trên Unix. Hàm này tạo một tiến trình mới và chỉ định chương trình mà tiến trình đó sẽ thực thi.
- WaitForSingleObject(): Tương đương với waitpid(). Hàm này cho phép tiến trình cha (Shell) chờ đợi tiến trình con (công việc tiền cảnh) kết thúc trước khi tiếp tục.
- TerminateProcess(): Dùng để buộc một tiến trình phải kết thúc, hữu ích trong việc xử lý tín hiệu.

Khi một lệnh được thực thi ở **chế độ tiền cảnh**, Shell sẽ:

1. Gọi CreateProcess().
2. Thêm tiến trình vào danh sách công việc với trạng thái FG.
3. Gọi WaitForSingleObject() để tạm dừng và chờ tiến trình con hoàn thành.
4. Sau khi tiến trình con kết thúc, xóa nó khỏi danh sách công việc.

Khi một lệnh được thực thi ở **chế độ nền**, Shell sẽ:

1. Gọi CreateProcess().
2. Thêm tiến trình vào danh sách công việc với trạng thái BG.
3. In ra thông tin về công việc nền (JID, PID).
4. Không chờ đợi, tiếp tục vòng lặp ngay lập tức để nhận lệnh mới.

e. Quản lý công việc nền (Jobs)

Quản lý công việc là một tính năng quan trọng. Tiny Shell cung cấp các lệnh để tương tác với các công việc đang chạy:

- **jobs**: Liệt kê tất cả các công việc đang chạy trong nền hoặc đang bị dừng, cùng với JID, PID, trạng thái và dòng lệnh của chúng.
- **bg <job>**: Chuyển một công việc đang bị dừng (ST) sang trạng thái chạy nền (BG). %<jid> hoặc <pid> được dùng để xác định công việc.
- **fg <job>**: Chuyển một công việc đang chạy nền (BG) hoặc đang bị dừng (ST) sang trạng thái chạy tiền cảnh (FG). Shell sẽ chờ công việc này hoàn thành.

Để làm được điều này, Shell cần duy trì một danh sách các công việc và trạng thái của chúng một cách chính xác.

f. Xử lý tín hiệu và sự kiện

Trên các hệ thống Unix, tín hiệu (signals) như SIGINT (Ctrl+C) và SIGTSTP (Ctrl+Z) là cơ chế chính để quản lý tiến trình. Trên Windows, cơ chế này khác biệt và thường được xử lý thông qua các sự kiện (events).

Để mô phỏng hành vi này trong Tiny Shell trên Windows, ta có thể sử dụng SetConsoleCtrlHandler(). Hàm này cho phép đăng ký một hàm callback (handler routine) sẽ được hệ điều hành gọi khi người dùng nhấn Ctrl+C, Ctrl+Break, v.v.

Trong hàm handler, Shell có thể:

- Xác định xem có công việc tiền cảnh nào đang chạy không.
- Nếu có, gửi một sự kiện (hoặc sử dụng GenerateConsoleCtrlEvent) để báo cho tiến trình con đó biết rằng nó cần phải kết thúc.
- Shell sẽ bắt và xử lý tín hiệu này để không tự kết thúc chính nó.

Việc triển khai đầy đủ tính năng này khá phức tạp và nằm ngoài phạm vi cốt lõi của một "Tiny" Shell cơ bản, nhưng đây là hướng tiếp cận đúng đắn.

4. PHÂN TÍCH CHI TIẾT MÃ NGUỒN

a. Các thư viện và hằng số

Mã nguồn sẽ bắt đầu bằng việc bao gồm các thư viện cần thiết và định nghĩa các hằng số.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>

#define MAXLINE 1024 // Kích thước tối đa của dòng lệnh
#define MAXARGS 128 // Số lượng đối số tối đa
#define MAXJOBS 16 // Số lượng công việc tối đa
```

b. Cấu trúc dữ liệu Job

Như đã đề cập ở phần thiết kế, cấu trúc `job_t` và mảng `jobs` là trung tâm của việc quản lý công việc.

```
// Các trạng thái của job
#define UNDEF 0 // Chưa xác định
#define FG 1 // Tiền cảnh
#define BG 2 // Nền
#define ST 3 // Dừng (mô phỏng)

struct job_t {
    DWORD pid; // Process ID (trên Windows là DWORD)
    int jid; // Job ID
    int state; // Trạng thái (FG, BG, ST)
    char cmdline[MAXLINE]; // Dòng lệnh
};

struct job_t jobs[MAXJOBS];
```

c. Các hàm tiện ích

Một số hàm tiện ích là cần thiết để thao tác với danh sách công việc:

- **clearjob(struct job_t *job)**: Khởi tạo lại một entry trong bảng job.
- **initjobs(struct job_t *jobs)**: Khởi tạo toàn bộ bảng job khi Shell khởi động.
- **maxjid(struct job_t *jobs)**: Tìm JID lớn nhất đang được sử dụng để gán JID mới.
- **addjob(struct job_t *jobs, DWORD pid, int state, char *cmdline)**: Thêm một công việc mới vào danh sách.
- **deletejob(struct job_t *jobs, DWORD pid)**: Xóa một công việc khỏi danh sách khi nó kết thúc.
- **getjobpid(struct job_t *jobs, DWORD pid)**: Tìm một công việc dựa trên PID.
- **getjobjid(struct job_t *jobs, int jid)**: Tìm một công việc dựa trên JID.

d. Xử lý lệnh tích hợp (Built-in Commands)

Hàm **builtin_cmd** đóng vai trò điều phối. Nó nhận các đối số đã được phân tích và kiểm tra xem đối số đầu tiên có phải là một trong các lệnh tích hợp hay không.

```
int builtin_cmd(char **argv) {
    if (!strcmp(argv[0], "quit")) {
        exit(0);
    }
    if (!strcmp(argv[0], "jobs")) {
        listjobs(jobs);
        return 1;
    }
    if (!strcmp(argv[0], "bg") || !strcmp(argv[0], "fg")) {
        do_bgfg(argv);
        return 1;
    }
    return 0; // Không phải lệnh built-in
}
```

Hàm **do_bgfg** sẽ chịu trách nhiệm phân tích đối số (ví dụ: **%1** hoặc **1234**), tìm công việc tương ứng và thay đổi trạng thái của nó.

e. Quản lý tiến trình

Phần cốt lõi nằm ở hàm thực thi lệnh ngoài. Trên Windows, nó sẽ sử dụng **CreateProcess**.

```

void eval(char *cmdline) {
    char *argv[MAXARGS];
    int bg;
    PROCESS_INFORMATION pi;
    STARTUPINFO si;

    // Thiết lập các cấu trúc STARTUPINFO và PROCESS_INFORMATION
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    bg = parseline(cmdline, argv);

    if (argv[0] == NULL)
        return; // Bỏ qua dòng trống

    if (!builtin_cmd(argv)) {
        // Tạo tiến trình con
        if (!CreateProcess(NULL, // Tên module
                           cmdline, // Dòng lệnh để thực thi
                           NULL, // Thuộc tính bảo mật tiến trình
                           NULL, // Thuộc tính bảo mật luồng
                           FALSE, // Không kế thừa handle
                           0, // Cờ tạo tiến trình
                           NULL, // Môi trường
                           NULL, // Thư mục hiện tại
                           &si, // Con trỏ đến STARTUPINFO
                           &pi)) // Con trỏ đến PROCESS_INFORMATION
        {
            fprintf(stderr, "CreateProcess failed (%d).\n", GetLastError());
            return;
        }

        if (!bg) { // Chạy tiền cảnh
            addjob(jobs, pi.dwProcessId, FG, cmdline);
            // Chờ tiến trình con kết thúc
            WaitForSingleObject(pi.hProcess, INFINITE);
            deletejob(jobs, pi.dwProcessId);
        } else { // Chạy nền
            addjob(jobs, pi.dwProcessId, BG, cmdline);
            printf("[%d] (%d) %s", getjobpid(jobs, pi.dwProcessId)->jid, pi.dwProcessId,
cmdline);
        }

        // Đóng các handle không cần thiết
        CloseHandle(pi.hProcess);
        CloseHandle(pi.hThread);
    }
}

```

}

f. Khởi động và quản lý Jobs

Các hàm như [addjob](#) và [deletejob](#) là xương sống của việc quản lý. [addjob](#) sẽ tìm một slot trống trong mảng [jobs](#) và điền thông tin của tiến trình mới vào. [deletejob](#) sẽ tìm công việc dựa trên PID và giải phóng slot đó.

g. Hàm [main](#)

Hàm [main](#) là điểm khởi đầu, chứa vòng lặp chính của Shell.

```
int main(int argc, char **argv) {
    char cmdline[MAXLINE];

    // Khởi tạo danh sách jobs
    initjobs(jobs);

    while (1) {
        printf("tsh> ");
        fgets(cmdline, MAXLINE, stdin);
        if (feof(stdin)) {
            exit(0);
        }

        // Đánh giá và thực thi dòng lệnh
        eval(cmdline);
    }
    return 0;
}
```

5. CÁC TÍNH NĂNG CHÍNH CỦA TINY SHELL

a. Thực thi lệnh cơ bản

Tính năng cơ bản nhất là khả năng thực thi các chương trình có sẵn trong hệ thống. Người dùng gõ tên chương trình cùng với các đối số, và Shell sẽ tạo một tiến trình mới để chạy nó.

- **Chế độ tiền cảnh (Foreground):** Đây là chế độ mặc định. Shell sẽ bị khóa và chờ cho đến khi lệnh thực thi xong.

```
tsh> notepad.exe mydocument.txt  
(Shell chờ cho đến khi Notepad đóng lại)  
tsh>
```

- **Chế độ nền (Background):** Bằng cách thêm dấu **&** vào cuối lệnh, người dùng yêu cầu Shell chạy lệnh trong nền. Shell sẽ ngay lập tức trở lại và sẵn sàng nhận lệnh mới.

```
tsh> calc.exe &  
[1] (5832) calc.exe &  
tsh>
```

b. Quản lý Jobs

Tiny Shell cung cấp một hệ thống quản lý công việc đơn giản.

- Lệnh **jobs**: Liệt kê tất cả các công việc đang chạy trong nền. Mỗi công việc được xác định bởi một Job ID (JID) duy nhất.

```
tsh> sleep.exe 100 &  
[1] (6124) sleep.exe 100 &  
tsh> notepad.exe &  
[2] (7832) notepad.exe &  
tsh> jobs  
[1] (6124) Running  sleep.exe 100 &  
[2] (7832) Running  notepad.exe &
```

c. Điều khiển tiến trình

Người dùng có thể thay đổi trạng thái của các công việc.

- Lệnh **fg** (Foreground): Đưa một công việc từ nền lên tiền cảnh. Shell sẽ bắt đầu chờ công việc này.

```
tsh> jobs  
[1] (6124) Running  sleep.exe 100 &  
tsh> fg %1  
(Shell bây giờ sẽ chờ cho sleep.exe hoàn thành)
```


- Lệnh **bg** (Background): (Trong một phiên bản đầy đủ hơn) Nếu một công việc bị dừng, lệnh **bg** sẽ làm nó tiếp tục chạy trong nền.

d. Các lệnh tích hợp

Đây là các lệnh được xử lý trực tiếp bởi Shell mà không cần tạo tiến trình mới.

- **quit**: Thoát khỏi Tiny Shell.
- **jobs**: Hiển thị danh sách các công việc.
- **fg <job>**: Chuyển job sang chế độ tiền cảnh.
- **bg <job>**: Chuyển job sang chế độ nền.

e. Xử lý môi trường

Mặc dù không phải là một tính năng được triển khai tường minh trong phiên bản cơ bản, Tiny Shell vẫn kế thừa môi trường từ Shell cha đã khởi chạy nó. Khi **CreateProcess** được gọi, tiến trình con mới sẽ tự động nhận một bản sao các biến môi trường của tiến trình Shell, cho phép các lệnh như **dir**, **echo %PATH%** hoạt động như mong đợi.

6. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

a. Các tình huống trong thử nghiệm

- Chạy chương trình background với ký tự &: ví dụ notepad &, calc &
- Liệt kê tiến trình nền: list để kiểm tra PID, trạng thái Running hoặc Stopped
- Tạm dừng và tiếp tục tiến trình nền: stop <id> rồi resume <id> và quan sát thay đổi
- Kết thúc tiến trình nền: kill <id> và xác minh không còn trong danh sách hoặc có trạng thái Exited
- Chạy file batch: build.bat hoặc bất kỳ file .bat có trong thư mục làm việc
- Lệnh hệ thống: date, time, dir để kiểm tra khả năng giao tiếp với hệ thống
- Kiểm tra PATH: path để hiển thị, addpath <dir> để nối thêm thư mục và gọi lại path xác minh

b. Kết quả đạt được

- Chạy background: notepad &, calc & khởi tạo tiến trình mới, được ghi vào bgList với trạng thái RUNNING
- list: hiển thị đúng chỉ số, lệnh gốc, PID và trạng thái của tiến trình nền

- stop <id> và resume <id>: tạm dừng và tiếp tục thread chính đã tạo bởi CreateProcess cho đa số chương trình console cơ bản. Với ứng dụng GUI đơn giản như Notepad, trạng thái tạm dừng/tiếp tục được phản ánh nhưng hành vi giao diện có thể không thay đổi rõ rệt
- kill <id>: tiến trình bị kết thúc, trạng thái được gán EXITED và không còn tương tác được
- .bat: gọi cmd.exe /c <file>.bat thực thi thành công các batch trong thư mục làm việc
- path và addpath <dir>: hiển thị PATH hiện tại và nối thêm thư mục mới cho phiên shell hiện tại. Gọi lại path thấy giá trị đã được cập nhật

c. Hạn chế và điểm cần cải thiện

***Hạn Chế:**

- Không theo dõi trạng thái thực tế của tiến trình nên sau khi khởi chạy. Biến status chỉ phản ánh hành động (stop/resume/kill) chứ không phát hiện được tiến trình tự thoát
- SuspendThread/ResumeThread chỉ áp dụng lên thread chính được CreateProcess tạo. Ứng dụng có nhiều thread hoặc tạo thread mới có thể không bị dừng hoàn toàn
- TerminateProcess là phương pháp kết thúc cưỡng bức, có thể gây rò rỉ tài nguyên hoặc dữ liệu chưa kịp ghi. Không có quy trình gửi tín hiệu thoát mềm (graceful)
- Không thu gom tiến trình con đã kết thúc để đóng handle và giảm rò rỉ tài nguyên. Chưa gọi WaitForSingleObject với thời gian chờ 0 để thăm dò trạng thái
- Không xử lý trường hợp MAX_BG đầy và không có tái sử dụng slot đã EXITED
- Không đồng bộ truy cập bgList trong trường hợp mở rộng lên đa luồng
- addpath chỉ có hiệu lực trong tiến trình hiện tại. Khởi chạy shell mới sẽ mất thay đổi PATH

***Điểm cần cải thiện:**

- Theo dõi và cập nhật trạng thái tiến trình nền bằng WaitForSingleObject(pi.hProcess, 0) theo chu kỳ. Khi thấy signaled thì gán EXITED và đóng handle
- Thêm lệnh fg <id> để đưa tiến trình nền ra foreground, và bg <id> để đẩy về nền. Cần nhắc FreeConsole/AttachConsole để gắn IO khi cần
- Thêm lệnh jobs hiển thị nhiều thông tin hơn: thời gian chạy, thời điểm bắt đầu, exit code nếu đã thoát

- Cài đặt kết thúc mềm: gửi WM_CLOSE cho cửa sổ GUI hoặc GenerateConsoleCtrlEvent(CTRL_BREAK_EVENT) cho tiến trình console trước khi dùng TerminateProcess như phương án dự phòng
- Gom dọn tài nguyên: đóng handle hProcess/hThread ngay khi không cần. Tạo hàm helper để đóng an toàn và gán NULL
- Tái sử dụng slot: khi EXITED thì cho phép ghi đè, hoặc chuyển sang cấu trúc vector động
- Bổ sung logic bảo vệ: kiểm tra lỗi của SuspendThread/ResumeThread, hiển thị GetLastError khi thất bại
- Hỗ trợ Unicode: dùng CreateProcessW và chuỗi wide để xử lý đường dẫn có ký tự Unicode
- Ghi log: thêm timestamp cho các sự kiện và tùy chọn bật tắt log qua biến môi trường
- Kiểm thử nâng cao: thêm batch tự động chạy các kịch bản và so sánh output mong đợi

8. TÀI LIỆU THAM KHẢO

a. Sách và tài liệu

1. *Operating System Concepts* - Abraham Silberschatz, Peter B. Galvin, Greg Gagne.
2. *Computer Systems: A Programmer's Perspective* - Randal E. Bryant, David R. O'Hallaron. (Tài liệu gốc cho bài lab tsh trên nền tảng Unix).

b. Nguồn trực tuyến

1. Microsoft Docs (MSDN): Tài liệu chính thức về Windows API, đặc biệt là các hàm CreateProcess, WaitForSingleObject, SetConsoleCtrlHandler.
 - o <https://docs.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-createprocessa>
2. Stack Overflow: Các cuộc thảo luận và giải pháp cho các vấn đề cụ thể gặp phải trong quá trình lập trình.